XR-PBD: A Cross-platform Physics Framework, with advanced Real-time Cutting, Tearing and Piercing of Tetrahedralised Soft-bodies in XR

ANONYMOUS AUTHOR(S)
SUBMISSION ID: PAPERS1350

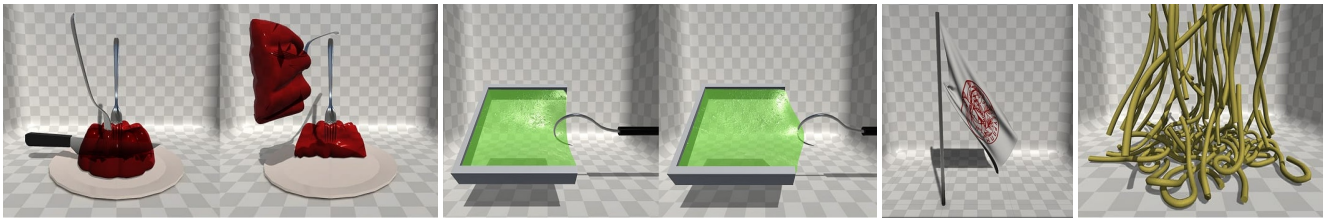

Fig. 1. Teaser figure demonstrating the capabilities of the XR-PBD framework for simulating realistic soft-body deformations and advanced interactions—cutting, tearing, and piercing—even on mobile head-mounted displays. Desktop VR was used for visualization to enhance image clarity. From left to right: (1) a jelly-like object undergoing cutting; (2) an elastic membrane pierced by a hook and deformed as it is stretched; (3) a waving flag simulation; and (4) multiple elastic rods ("spaghetti") falling and deforming upon impact.

We introduce XR-PBD, a novel real-time Position-Based Dynamics (PBD) framework tailored for mobile XR applications. XR-PBD enables unified simulation of cutting, tearing, and piercing of deformable tetrahedralized sof-bodies in real time, addressing core challenges in dynamic topological modification. Our method integrates a novel insertion constraint formulation for realistic needle-surface interactions and supports dynamic remeshing, constraint reassembly, and topological adaptivity. We demonstrate significant performance gains over state-of-the-art methods, with up to 39% improvement in physics updates and 230% in render updates, while preserving simulation quality on resource-constrained mobile devices. XR-PBD is validated through benchmarks and representative surgical and textile XR scenarios. This framework also establishes the first step toward a scalable synthetic data generator for training neural network simulators capable of learning and replicating complex topological operations such as cutting and piercing. This dual focus positions XR-PBD not only as practical tool for advanced real-time interaction, but also as a foundational platform for physics-informed machine learning in XR simulation.

CCS Concepts: • **Computing methodologies** → *Mesh geometry models*; Virtual reality; • **Mathematics of computing** → Mesh generation.

Additional Key Words and Phrases: Real-time simulation, Position-Based Dynamics (PBD), Cut Tear Pierce, Deformable objects, Extended Reality (XR), Interactive physics, GPU acceleration, Constraint-based modeling

ACM Reference Format:
Anonymous Author(s). 2025. XR-PBD: A Cross-platform Physics Framework, with advanced Real-time Cutting, Tearing and Piercing of Tetrahedralised Soft-bodies in XR. *ACM Trans. Graph.* 1, 1 (May 2025), 8 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

ACM 0730-0301/2025/5-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Interactive physics-based simulations are increasingly central to modern XR applications, enabling more immersive training, learning, and entertainment scenarios. Yet, despite rapid advancements in physics engines and real-time simulation paradigms, supporting physically-plausible, high-frequency interactions—such as cutting, tearing, and piercing—remains a major challenge, particularly on mobile XR platforms constrained in terms of CPU and GPU.

Real-time handling of such interactions typically requires compromise: most existing systems either restrict interaction types to a narrow subset (e.g., only cutting or only puncture), rely on offline preprocessing (e.g., pre-fractured meshes), or assume availability of high-end GPU hardware. This imposes severe limitations on interactivity, realism, and scalability—particularly for use cases such as surgical simulation, material simulation , or educational exploration, where granular, physically-responsive behaviors are essential.

To address these limitations, we present *XR-PBD*, a unified cross-platform physics framework for advanced real-time interactions with deformable objects, specifically designed to operate within the performance envelope of mobile XR. Our method enables dynamic cutting, tearing, and piercing of soft deformable bodies in a single integrated architecture, without requiring any precomputed topology or material configurations. The approach is built on an extended Position-Based Dynamics (XPBD) [Macklin et al. 2016] solver and incorporates a modular constraint system, material-aware detectors, and runtime remeshing support—all optimized for mobile hardware.

The key contributions of this work are:

(1) A unified interaction framework supporting advanced mesh operations on deformable surfaces, such as cutting, tearing, and piercing, within a single, coherent simulation loop;
(2) A modular detector-constraint pipeline to capture and resolve high-frequency topological changes in real time;
(3) A set of runtime optimizations enabling efficient execution on commercial mobile XR headsets, including adaptive remeshing and parallel solver enhancements;

(4) A quantitative evaluation of interaction fidelity, application latency, and stability, along with demonstrative use cases in untethered mobile VR.

This framework aims to enable the next generation of interactive applications with rich material responses and real-time topological interactions, supporting advances in immersive simulation and XR training on lightweight hardware. XR-PBD also serves as a first step toward a synthetic data generator for training neural netwprk simulators of surgical procedures involving cutting tearing and piercing. By combining this data with physics-informed machine learning, we aim to develop fast, learning-based models that retain physical realism - opening new possibilities for scalable, data-driven surgical simulation in mobile XR.

2 RELATED WORK

2.1 Deformable Simulation and Position-Based Methods

Deformable object simulation has long been a cornerstone of physics-based modeling in graphics. Early methods relied on mass-spring systems or finite element models (FEM) [Müller et al. 2005], which, while accurate, often require small timesteps and heavy computational resources. In contrast, *Position-Based Dynamics (PBD)* [Bender et al. 2015] and its extension, *XPBD* [Macklin et al. 2016], trade physical accuracy for real-time stability by operating directly on positions. These methods are particularly attractive for interactive applications and mobile deployment due to their unconditional stability and modular constraint handling. Our framework builds on XPBD, extending it to support dynamic topological changes and high-frequency fracture behavior.

2.2 XR Physics Engines and Simulation Toolkits

In this section, we review prior physics simulation frameworks, focusing on their capabilities and limitations relevant to real-time softbody and interactive dynamics.

Nvidia PhysX [noa 2025] is a widely adopted physics engine that employs a velocity-based solver for rigid body dynamics and a position-based solver for increased stability in cloth and softbody simulations. It is integrated into Unity with support for rigid bodies, joints, and cloth, but lacks native softbody support and real-time interactions like cutting or tearing. Hardware acceleration depends on CUDA [NVIDIA [n. d.]], which is unavailable on mobile XR Head-mounted Displays (HMDs), limiting its applicability in XR settings.

Unity Physics [Unity [n. d.]] is a stateless, deterministic engine designed for the DOTS architecture, leveraging the Burst compiler for parallel CPU execution. It utilizes a velocity-based solver and offers a lightweight, customizable physics stack, omitting support for softbodies or advanced interactions like tearing or cutting. As such, developers are expected to extend its modular core to implement these features.

Nvidia Flex [NVIDIA 2025] is a unified particle-based framework using Extended Position Based Dynamics for stable simulations of rigid bodies, softbodies, fluids, and smoke. Despite its versatility and performance, it is tied to CUDA [NVIDIA [n. d.]] and no longer actively maintained, as Nvidia has shifted focus to PhysX. Flex also lacks support for interactive operations like cutting, tearing or piercing.

Obi Physics [Studio 2025] offers a suite of Unity-native packages for softbodies, fluids, cloth, and ropes, optimized via the Burst Compiler and GPU Compute shaders. While user-friendly and robust for deformable simulations, it does not support advanced interactions such as cutting tearing and piercing, which are integral to XR surgical training applications.

The SOFA framework [Faure et al. 2012] is an open-source, modular simulation platform tailored to research applications, enabling complex setups including FEM and softbody dynamics. Despite its flexibility, SOFA requires a steep setup curve involving proprietary scene formats and has limited Unity integration. It is not compatible with Android, restricting its use on XR HMDs.

The iMSTK framework [Moore et al. 2023] enables advanced real-time simulations with support for PBD, continuous collision detection, and haptic feedback. Although it supports interactions like piercing and is compatible with major engines, its C++ codebase and minimal Unity integration hinder usability.

2.3 Advanced 3D Mesh Operations: Cutting and Tearing

This section reviews prior work addressing real-time user interactions for advanced mesh operations such as cutting and tearing of deformable objects in virtual environments. Cutting typically relies on mesh subdivision or volumetric remeshing [Bielser and Gross 2000; Steinemann et al. 2006] , while tearing and puncture are often modeled via node disconnection [Paulus et al. 2015]. However, most prior methods are limited to *offline use* or high-end computing environments [Kokiadis et al. 2024]. Real-time fracture in XR remains underexplored, with systems typically addressing only a *single interaction type*. For example, Unity and PhysX-based engines may support basic puncture or mesh slicing [NVIDIA 2025], but lack generality, extensibility, or robustness across interaction modes.

A series of studies [Kamarianakis et al. 2021, 2022a,b] proposed a real-time framework for tearing and cutting soft bodies within VR environments. The deformable meshes were modeled using a spring-mass system operating on surface meshes. Cutting operations were realized by intersecting a bounded planar blade with the surface mesh, defining a cutting path where mesh triangles were split along intersection points. New particles were inserted along the cut to enhance collision handling and reduce artifacts. This approach demonstrated real-time performance on low-spec mobile head-mounted displays, enabling interactive simulations. However, the method was limited to surface meshes, resulting in cuts that lacked volumetric depth, thereby restricting applicability to fully deformable volumetric objects. Subsequent extensions integrated these techniques into a VR training software development kit [Zikas et al. 2023], improving interaction capabilities.

Sifakis et al. [Sifakis et al. 2007] introduced a sophisticated algorithm for cutting deformable objects represented by tetrahedral meshes. Their method utilized a dual-mesh approach, employing a low-resolution simulation mesh for physics computations and a higher-resolution render mesh for visualization. Cutting surfaces were triangulated, and intersections with both meshes identified points where new virtual nodes were created. Instead of directly

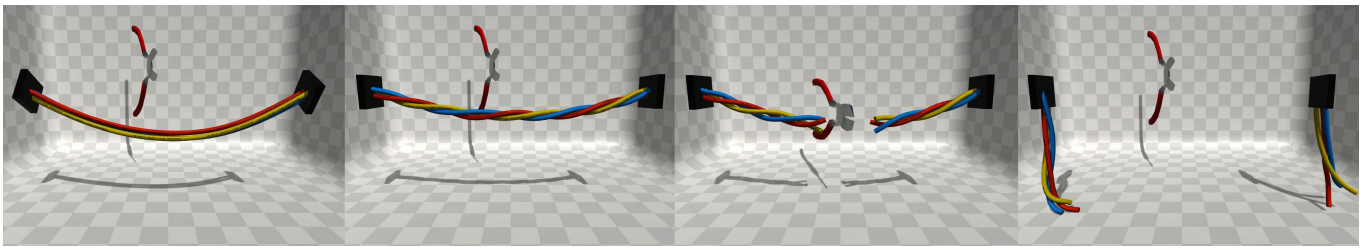

Fig. 2. Simulation and cutting of Twisted cables, i.e., deformable soft-body rods.

splitting intersected tetrahedra, the method duplicated them to prevent the formation of sliver and ill-conditioned elements that commonly cause simulation instabilities. Physical deformation was simulated via the simulation mesh vertices, while collision detection relied on the render mesh's surface triangles. Although this approach enables high-quality volumetric cutting, its complexity hinders real-time performance on reasonably sized meshes, particularly on resource-constrained devices such as mobile XR HMDs. Maintaining two separate mesh structures also increases memory overhead and complicates data management. Additionally, collision detection based on the render mesh can suffer due to small and ill-conditioned triangles, as noted by the authors.

3 XR-PBD FRAMEWORK OVERVIEW

This paper proposes, a custom physics simulation framework, called XR-PBD (Extended Reality - Position Based Dynamics), tailored for performance-constrained devices such as standalone XR HMDs. Leveraging the power and stability of Extended Position Based Dynamics (XPBD)[Macklin et al. 2016] our method enables real-time simulation of rigidbodies, soft-bodies, cloths, and ropes. Through the introduction of new constraints types, our framework enables advanced user interactions such as real-time cutting (e.g., Figure 5), tearing (e.g., Figure 6) and piercing (Figure 1, second from the left). Designed for low-spec XR HMDs, XR-PDB EMPLOYES a massively parallel architecture, optimized for multi-core CPUs and an advanced memory layout. Built around Unity Engine, it integrates no-code simulation editing and authoring tools (see Section 4.4), enabling precise control and validation of interactive scenarios without requiring low-level scripting.

As a high-level overview of XR-PBD's core architecture, the framework is designed around a particle system whose state evolves under a set of geometrically motivated constraints. During interactions with the tetrahedralised meshes in the scene, these constraints are continuously solved by a dedicated solver, while collision handling and user input are processed in parallel. At each simulation frame, the updated particle states are evaluated, and the resulting mesh is rendered. The following sections describe the fundamental components of this process: particles, constraint formulations, and the parallel solver.

Particle System: Each deformable object is discretized into a set of particles $X = \{\mathbf{x}_i\}_{i=1}^{N}$, storing position, mass, and velocity. We employ structured data arrays to maximize coherence and computation speed. Both surface-based and volumetric representations are supported.

Constraint Formulations: Constraints enforce deformation behavior, via constraint functions $C(X)$ [Macklin et al. 2016]:

- **Distance**: Maintain rest length between particles.
- **Shape Matching**: Prevents particle displacement.
- **Volume**: Enforce local volume invariance.
- **Collision**: Prevent particle interpenetration.

Parallel Successive Over-Relaxation Solver: To leverage the parallelism of modern multi-core CPUs, we introduce a Jacobi-style constraint solver enhanced with Successive Over-Relaxation (SOR). Unlike traditional Gauss-Seidel approaches used in Position-Based Dynamics (PBD), which process constraints sequentially, our solver accumulates corrections in parallel and applies them collectively, enabling efficient multithreaded execution. To compensate for the slower convergence of Jacobi methods, we scale the aggregated corrections using a global SOR factor $s \in [1, 2]$, improving convergence while preserving compatibility with dynamic mesh topologies that arise from real-time cutting and tearing.

Table 1. Comparison of physics frameworks. Abbreviations in header: **R**: Rigids, **S**: Softs, **C**: Cloths, **RR**: Ropes & Rods, **CTP**: Cut Tear Pierce, **CP**: Cross Platform, **NC**: No-Code, **Per**: Performance. Notation: (✓) fully supported, (✓✗) partially supported, (✗) not supported.

Name	R	S	C	RR	CTP	CP	NC	Per
Unity PhysX	✓	✗	✓	✗	✗	✓	✓	High
Unity Physics	✓	✗	✗	✗	✗	✓	✓	High
NVIDIA Flex	✓	✓	✓	✓	✓✗	✗	✓✗	High
Obi Physics	✗	✓	✓	✓	✓✗	✓	✓	High
SOFA	✓	✓	✓	✓✗	✓	✗	✓✗	Low
imstk	✓	✓	✓	✓✗	✓	✗	✓✗	Medium
XR-PBD (Ours)	✓	✓	✓	✓	✓	✓	✓	High

4 TECHNICAL CONTRIBUTIONS

4.1 Insertion Constraint

To support real-time piercing interactions we propose a new type of constraint, the Insertion Constraint. It involves five particles: three form the triangle that is punctured at a point q with barycentric coordinates $u = (u_x, u_y, u_z)$, and the other two define a line segment -part of the needle used for piercing- from now on referred to as "needle segment". The constraint is described as:

$$C(x_1, x_2, x_3, n_1, n_2) = |q - r| \tag{1}$$

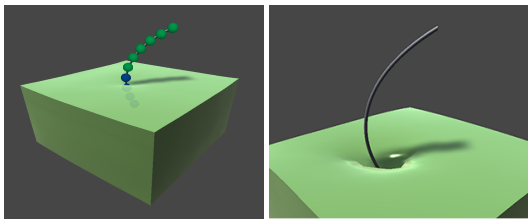

Fig. 3. **Left:** Collisions of particles that are inside the pierced softbody and the next particle from the ones outside are disabled (shown with blue). **Right:** High friction resists the needle's penetration into the softbody.

where $q = u_x x_1 + u_y x_2 + u_z x_3$ and $r = n_1 + t(n_2 - n_1)$, with r denoting the point of intersection between the needle and the triangle, for some proper $t \in \mathbb{R}$.

To derive the constraint gradients we will assume that t is constant between timesteps. Although this is not entirely accurate, the change in t between timesteps is negligible, small enough that the resulting error is imperceptible in the final outcome and does not justify the added computational complexity of treating t as a variable.

The gradient for each particle are calculated as:

$$\nabla C(x_1, x_2, x_3, n_1, n_2) = \frac{q - r}{|q - r|}(u_x, u_y, u_z, (t-1), t), \qquad (2)$$

which is used at each iteration of the solver.

4.2 Piercing Mechanics and Constraints

XR-PBD introduces additional constraints to enable piercing interactions with watertight softbody objects. To simulate the piercing process, we use a rigidbody referred to as the "needle", constructed by placing a series of particles along a curve that approximates the needle shape. These adjacent particles are connected to form the needle segments, with the first particle in the simulation mesh representing the needle tip.

Prior to simulation, exterior triangle faces of the softbody are identified and stored as reference surfaces for intersection detection.

At runtime, after each simulation timestep, intersections between needle segments and tetrahedra faces, defined by volume constraints, are checked, starting from the tip. An intersection is considered valid only if both the current and previous segments intersected the softbody in consecutive steps. In that case, an insertion constraint is created involving the three particles of the intersected triangle and the two particles of the needle segment (Figure 4(a)).

To avoid incorrect collisions while the needle is inside the softbody, collision detection is selectively disabled (Figure 3). Needle entry is detected via swept sphere tests against precomputed exterior faces. If a particle crosses to the opposite side of a face normal, collisions are disabled; they are re-enabled once the particle returns to the original side. Collisions for the leading needle particles outside the softbody volume are also disabled to prevent interference during insertion.

With low iteration counts, insertion constraints may remain unsatisfied at the end of a timestep, allowing the needle slip and lose intersection with the triangle, causing premature constraint removal (Figure 4(b)). To prevent this, constraints are retained until a needle

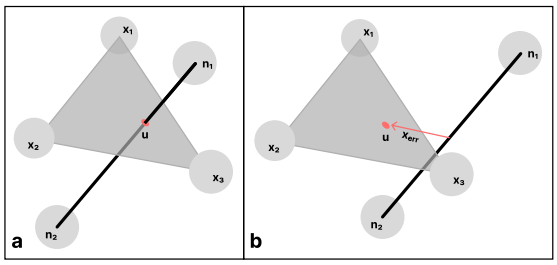

Fig. 4. a) Insertion constraint involving triangle particles x_1, x_2, x_3 and needle segment particles $n_1, n_2$with puncture point u n barycentric coordinates. b) Constraint incorrectly removed when intersection is lost at the end of the timestep.

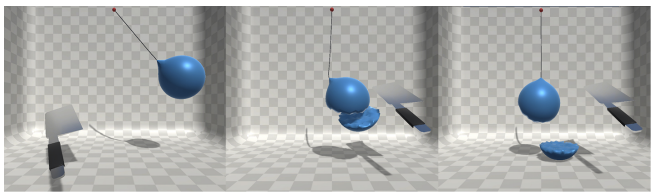

Fig. 5. Timelapse of a ball hanging from a rope being cut by a cleaver.

particle pierces the triangle's plane again, detected via the same swept sphere test without extra overhead.

Friction constraints are applied alongside the insertion constraints to limit needle movement by reducing relative motion at the insertion point u, based on a predefined friction coefficient.

4.3 Cutting and Tearing

XR-PBD supports two cutting strategies for deformable soft bodies: removal-based and split-based cutting. Both rely on detecting intersections between a virtual cutting disc and the simulation mesh. In the removal-based approach, intersected volume constraints are deleted, allowing the mesh to separate freely. In contrast, the split-based method bisects intersected volume constraints and re-tetrahedralizes the resulting subdomains, yielding a more anatomically precise incision. A post-processing step follows to eliminate residual constraints and sliver tetrahedra, maintaining simulation stability. For one-dimensional structures such as ropes and rods, XR-PBD applies a specialized variant that removes intersected distance and collinear constraints, followed by a reconstruction of the render mesh.

Tearing in XR-PBD is integrated directly into the constraint-solving loop, allowing it to emerge dynamically from the simulation without reliance on predefined fracture paths. After each solver iteration, the method estimates the force on each particle using the relation $F = \frac{\Delta x}{\Delta t}$, where Δx denotes the positional correction and Δt is the simulation timestep. If the computed force exceeds a user-defined threshold, the associated constraint is deactivated, resulting in a tear. This physically grounded approach enables material rupture to occur adaptively in response to stress, enhancing realism and flexibility in interactive applications.

4.4 No-Code Abstractions for XR-PBD in Unity

While XR-PBD achieves real-time performance with visually convincing results, its integration in game engines can introduce significant complexity, particularly due to low-level memory management and detailed configuration procedures. To address this, a no-code abstraction layer is introduced, consisting of **Actors** and **Detectors**, which encapsulate core simulation logic and event monitoring within reusable high-level components. This architecture enables streamlined integration with game engine-based workflows while maintaining full simulation capabilities.

Actors represent high-level simulation primitives that abstract the underlined Entity-Component-System (ECS)-inspired structure. They expose both physics and rendering functionality through a simplified interface. XR-PBD supports six Actor types: Rigidbody (shape matching for near-rigid bodies), Softbody (decoupled render/simulation meshes with support for piercing), Dynamic Softbody (real-time tearing with automatic mesh regeneration), Rope (1D particle chains with stretch and bend constraints), and Cloth (2D sheets supporting tearing and piercing). All Actors (Figures 1, 2) are integrated with the Unity Editor, enabling visual configuration and simulation authoring without scripting.

To support real-time monitoring of changes in XR-PBD simulations, *Detectors* are introduced as modular components that monitor and expose key events such as punctures, tears, constraint failures, and floor collisions. The system includes several prebuilt Detectors: Particle Disconnection Detector (monitors topological separation via graph traversal), Constraint Destruction Detectors (track disabled distance or volume constraints), Puncture Detector (detects insertion events in softbodies), and Floor Collision Detector (signals contact with the ground). Each Detector is accompanied by Unity Editor tools for intuitive parameter tuning configuration. For custom scenarios, an extensible Base Detector class is provided, offering structured access to the mesh and simulation state without imposing low-level overhead.

5 IMPLEMENTATION AND OPTIMIZATION

XR-PBD was designed to operate under the stringent constraints of real-time performance, high visual fidelity, and topological flexibility required by immersive XR applications. In this section, we present a detailed account of the system's architecture, data layout, constraint resolution mechanisms, and topological operations, with an emphasis on the strategies employed to maintain high frame rates under conditions of continuous user interaction and dynamic mesh modification.

5.1 Main Simulation Loop

The primary simulation loop in our SOR Solver, manages the integration of the physics simulation for each time step.First the Collision World is updated and the contact pairs between colliding particles are collected. The updated velocities are then computed, after which damping is applied and the new particle positions are predicted. Subsequently, constraints are resolved, and the average position correction, scaled by the relaxation factor, is applied to the predicted particle positions. Finally, the updated velocity of each particle is determined, which dictates whether the particle enters a sleeping state or adopts the predicted position.

5.2 Structure of Arrays

To optimize computational performance, our implementation adopts a *Structure of Arrays* (SoA) data organization model rather than the conventional *Array of Structures* (AoS). This design choice aligns with the optimization heuristics employed by Unity's Burst Compiler and Job System. By storing each data attribute (e.g., positions, velocities, masses) in contiguous memory blocks, SoA significantly enhances data locality and minimizes cache misses. This layout facilitates SIMD-friendly memory access patterns and enables the Burst Compiler to leverage vectorized instructions, thereby achieving substantial speedups on modern CPU architectures.

5.3 Constraint Resolution and Parallel Solver Design

The core solver is based on PBD [Müller et al. 2007], using a Jacobi-style solver to exploit GPU parallelism while maintaining adequate constraint convergence. We implemented a constraint graph scheduler that partitions the set of constraints into non-conflicting batches, ensuring that parallel evaluations do not overwrite shared particle states. Each constraint is processed via a compute shader dispatch that resolves position corrections and writes them to a shared accumulation buffer. These corrections are applied atomically using warp-wide synchronization primitives, ensuring determinism within a frame. Multiple solver iterations (typically 6–10) are executed per frame to enhance stability and compliance. To handle high stiffness constraints (e.g., in regions near incisions), we employ constraint warm starting, where residual corrections from the previous frame are blended into the current iteration, accelerating convergence.

5.4 Sleeping Mechanism

To further optimize computation, the framework implements a *sleeping* mechanism that selectively deactivates particles and constraints exhibiting negligible motion or forces over consecutive frames.

Sleeping particles are excluded from solver updates until reactivated by interactions or external forces, thereby reducing the active simulation set and saving processing power without affecting visual or physical accuracy.

5.5 Adaptive Remeshing

Adaptive remeshing focuses computational resources where high deformation and interaction occur. Our method:

- Locally refines tetrahedral meshes near cutting or tearing sites to maintain accuracy.
- Coarsens regions exhibiting low deformation to reduce particle count.
- Updates connectivity and constraint sets incrementally to avoid full mesh reconstruction.

This spatial adaptivity balances simulation quality and efficiency, making the framework suitable for mobile XR applications with limited compute budgets.

5.6 Optimizations towards real-time performance even in Mobile XR Headsets

To achieve optimal performance, particularly on low-spec HMDs, we employed state-of-the-art additional runtime optimization techniques, including Unity's Job System, Burst Compiler and a SoA approach for data organization.

The Unity Job System, a multithreading framework, allowed us to design highly parallelized code in a structured and efficient manner. By enabling tasks to execute independently across multiple CPU cores, it maximizes the performance potential of modern hardware. This framework is built around the concept of jobs, self-contained units of work that can be scheduled and executed asynchronously. Adopting a job-based architecture allowed us to offload computationally intensive tasks, such as physics calculations, from the main thread, significantly reducing bottlenecks and improving overall application performance.

Furthermore, the Burst Compiler was utilized to complement the Job System, providing additional layers of optimization. The Burst Compiler is a high-performance compiler integrated into Unity, designed to translate managed C# code into highly optimized assembly. Burst enhances performance through advanced optimizations, such as, use of SIMD instructions (Single instruction,multiple data), vectorization, inlining, and efficient memory layout adjustments, tailored to modern processor architectures. By eliminating unnecessary abstraction layers and Just In Time(JIT) compilation, it minimizes runtime overhead and ensures deterministic and predictable execution, making it especially effective for computationally intensive operations.

Finally, we have implemented a unified cutting and tearing system that dynamically destroys constraints during topological changes. This process is optimized by efficiently flagging and removing invalidated constraints from solver data structures, minimizing memory fragmentation through pool allocators, and deferring expensive constraint updates/rebuilds to preserve a consistent solver state with minimal overhead. Such dynamic management is crucial for maintaining real-time responsiveness on resource-constrained devices.

6 RESULTS

This section presents a quantitative and qualitative evaluation of the XR-PBD framework focusing on piercing, tearing, and cutting operations. We compare our solver's performance and visual fidelity against the SOFA framework [Faure et al. 2012], a state-of-the-art deformable simulation platform, to demonstrate the advantages of our unified approach.

All benchmarks were conducted on a representative mobile XR device (specifications: ARM Cortex-A78 CPU, 8 cores, 2.6 GHz, 8 GB RAM, Adreno 650 GPU) to assess real-time feasibility. Simulations used tetrahedral meshes with particle counts ranging from 2,000 to 10,000, typical for interactive soft tissue models in XR.

6.1 Performance Evaluation

The XR-PBD framework consistently outperforms SOFA, particularly as mesh complexity increases, due to its parallel SOR solver and adaptive remeshing optimizations. Figure 7 shows a 39% improvement in physics update FPS and a 230% improvement in render update FPS compared to the SOFA framework.

Table 2. Average time needed for a time step on a portable Meta Quest 2 HMD.

Scene	Particles	Mesh Update (ms)	Render Update (ms)	Total (ms)
Twisted Cables	50	2.76	0.25	3.01
Ball Cutting	225	6.25	0.05	6.3
Hook Pulling	484	9.64	0.05	9.69
Falling Rods	912	2.66	0.14	2.8
Flag	1089	10.05	0.05	11
Jelly	1815	18.09	0.05	18.13
Jelly Tearing	1815	14.98	0.04	15.02
Falling Rigids	8850	17.05	-	17.05

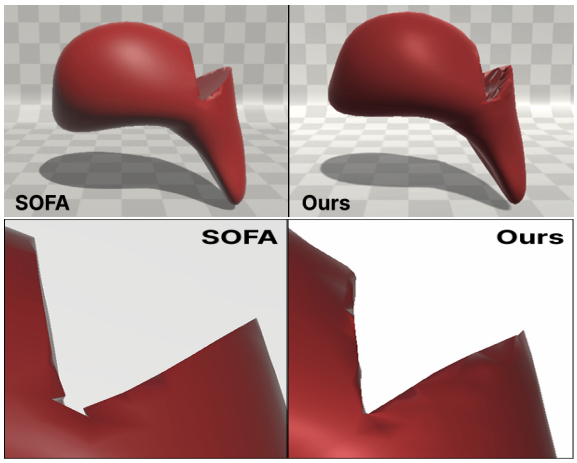

Fig. 6. Visual comparison of cutting and tearing simulations between SOFA (left) and our method, XR-PBD (right). XR-PBD exhibits smoother mesh continuity and precise topological changes.

6.2 Visual Comparisons

Figure 6 presents side-by-side visual comparisons between XR-PBD and SOFA during complex cutting and tearing scenarios. Our framework maintains high-fidelity mesh deformation and smooth topological updates without artifacts commonly observed in SOFA, such as mesh distortions and delayed constraint updates.

6.3 Discussion and Comparison with State-of-the-Art

The results demonstrate that XR-PBD's unified solver architecture and optimization techniques not only achieve significant performance gains but also improve simulation robustness during topological changes. The ability to perform piercing, tearing, and cutting

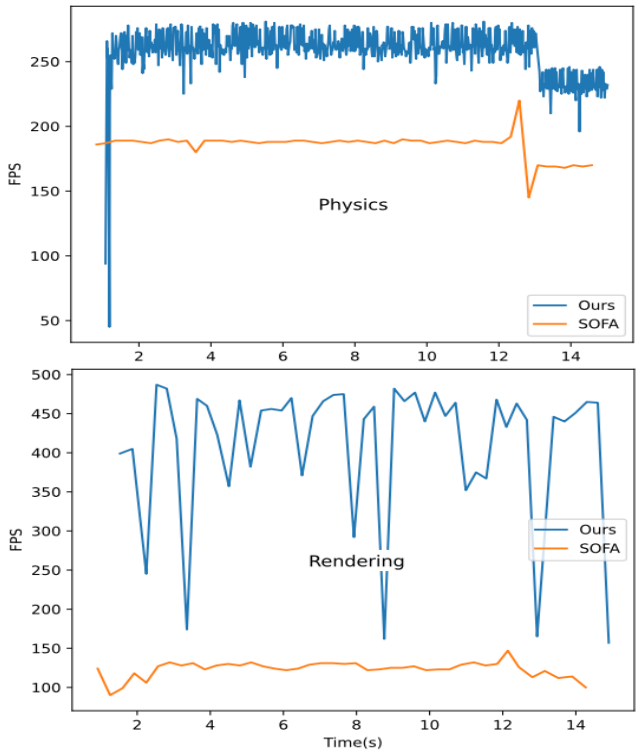

Fig. 7. Performance comparison with SOFA. Top: Physics Update FPS 39% faster. Bottom: Render Update FPS 230% faster. Spikes in measurements in our framework originate from Garbage Collection tasks and Job scheduling logic executed in main thread before a solver loop starts.

within a single solver framework, coupled with adaptive remeshing, enables real-time interactions on mobile XR devices without compromising visual realism or physical accuracy.

Table 1 presents a comparative overview of physics frameworks, highlighting XR-PBD as the only method offering full support for deformables (softs, cloths, ropes), advanced topological operations (cutting, tearing, piercing), and cross-platform, no-code integration. Unlike existing solutions, that typically compromise on at least one axis, XR-PBD uniquely combines broad feature support with high runtime performance.

Compared to SOFA, the proposed method offers native integration within the Unity engine, cross-platform operating system compatibility, and an editor-based workflow that eliminates the need for external scene description formats. In contrast to iMSTK, XR-PBD provides a modular authoring experience directly within Unity and extends functionality to support real-time cutting and tearing.

6.4 Toward Physics-Informed Learning Frameworks

While XR-PBD was primarily designed as a real-time position-based dynamics framework for interactive topological modifications, its architecture also lays the groundwork for scalable data-driven simulation. Specifically, the modular constraint solver, unified treatment of cutting, tearing, and piercing, and fine-grained temporal resolution make it well-suited for generating high-quality synthetic datasets. These datasets can serve as training corpora for neural network-based surrogate models that aim to approximate or accelerate physical simulation. We envision future extensions of XR-PBD enabling supervised learning of differentiable simulators that can infer constraint behavior or predict topological changes directly from state histories. Although preliminary, our efforts in this direction mark a dual trajectory for XR-PBD—as both a practical runtime engine for advanced user interaction and a foundational platform for physics-informed machine learning in XR environments.

6.5 Limitations

Currently, relying solely on particle-particle collisions, while computationally efficient, is not well-suited for scenarios where the interacting objects differ significantly in scale. The collision detection and response system, although quite responsive, can be further enhanced by introducing additional collision shapes, such as primitives (e.g., cubes, spheres, and capsules). This will enable smaller rigid objects to precisely collide with larger softbodies.

Moreover, the rigid body simulation is not perfect; an even more detailed simulation would require the incorporation of further shape-matching constraints. Using advanced joint constraints would significantly enhance interactions with rigid tools and eventually allow for complex multi-part rigidbody simulation.

7 CONCLUSIONS AND FUTURE WORK

This work presented a unified real-time framework for cutting, tearing, and piercing in deformable soft bodies, optimized for mobile XR platforms. Leveraging an extended position-based dynamics formulation, advanced constraint handling, and adaptive remeshing, the system achieves high-fidelity deformation with interactive performance suitable for untethered VR applications such as surgical training and material exploration.

Future work will focus on several directions to further enhance the framework's capabilities and scalability. A primary extension involves porting the solver to GPU architectures via Compute Shaders to exploit massively parallel computation, significantly improving simulation throughput and enabling more complex scenes. Moreover, integrating physics - informed machine learning models offers potential to accelerate constraint solving and improve predictive accuracy by learning material behaviors from data.

To support this vision, XR-PBD will be extended into a scalable synthetic data generation pipeline tailored for training neural network simulators. This includes automated dataset creation with ground-truth annotations of topological operations—such as cutting, tearing, and piercing—which are otherwise difficult to acquire at scale. This dual focus positions XR-PBD not only as a practical tool for advanced real-time interaction, but also as a foundational platform for physics-informed learning in XR simulation.

Other avenues include extending the framework to support heterogeneous, multi-material interactions, incorporating more sophisticated fracture models, and enhancing user interaction modalities to increase immersion and usability. Collectively, these developments aim to establish a comprehensive platform for next-generation mobile XR simulations with broad applicability across education, research, and industrial domains.

REFERENCES

2025. NVIDIA-Omniverse/PhysX. https://github.com/NVIDIA-Omniverse/PhysX original-date: 2022-10-04T09:07:32Z.

Jan Bender, Matthias Müller, and Miles Macklin. 2015. Position-Based Simulation Methods in Computer Graphics.. In *Eurographics (tutorials)*. 8.

D. Bielser and M.H. Gross. 2000. Interactive simulation of surgical cuts. In *Proceedings the Eighth Pacific Conference on Computer Graphics and Applications*. IEEE Comput. Soc, Hong Kong, China, 116–442. https://doi.org/10.1109/PCCGA.2000.883933

Francois Faure, Christian Duriez, Hervé Delingette, Jeremie Allard, Benjamin Gilles, Stéphanie Marchesseau, Hugo Talbot, Hadrien Courtecuisse, Guillaume Bousquet, Igor Peterlik, and Stéphane Cotin. 2012. SOFA: A Multi-Model Framework for Interactive Physical Simulation. Vol. 11. https://doi.org/10.1007/8415_2012_125

Manos Kamarianakis, Nick Lydatakis, Antonis Protopsaltis, John Petropoulos, Michail Tamiolakis, Paul Zikas, and George Papagiannakis. 2021. "Deep Cut": An all-in-one Geometric Algorithm for Unconstrained Cut, Tear and Drill of Soft-bodies in Mobile VR. https://doi.org/10.48550/arXiv.2108.05281 arXiv:2108.05281 [cs].

Manos Kamarianakis, Antonis Protopsaltis, Dimitris Angelis, Michail Tamiolakis, and George Papagiannakis. 2022a. *Progressive Tearing and Cutting of Soft-bodies in High-performance Virtual Reality*. The Eurographics Association. https://doi.org/10.2312/egve.20221275 ISSN: 1727-530X.

Manos Kamarianakis, Antonis Protopsaltis, Michail Tamiolakis, and George Papagiannakis. 2022b. Realistic soft-body tearing under 10ms in VR. https://doi.org/10.48550/arXiv.2205.00914 arXiv:2205.00914 [cs].

George Kokiadis, Antonis Protopsaltis, Michalis Morfiadakis, Nick Lydatakis, and George Papagiannakis. 2024. Decoupled Edge Physics algorithms for collaborative XR simulations. *Computer Animation and Virtual Worlds* 35, 6 (2024), e2294.

Miles Macklin, Matthias Müller, and Nuttapong Chentanez. 2016. XPBD: position-based simulation of compliant constrained dynamics. In *Proceedings of the 9th International Conference on Motion in Games*. ACM, Burlingame California, 49–54. https://doi.org/10.1145/2994258.2994272

Jacob Moore, Harald Scheirich, Shreeraj Jadhav, Andinet Enquobahrie, Beatriz Paniagua, Andrew Wilson, Aaron Bray, Ganesh Sankaranarayanan, and Rachel B. Clipp. 2023. The interactive medical simulation toolkit (iMSTK): an open source platform for surgical simulation. *Frontiers in Virtual Reality* 4 (June 2023). https://doi.org/10.3389/frvir.2023.1130156 Publisher: Frontiers.

Matthias Müller, Bruno Heidelberger, Matthias Teschner, and Markus Gross. 2005. Meshless deformations based on shape matching. *ACM transactions on graphics (TOG)* 24, 3 (2005), 471–478.

Matthias Müller, Bruno Heidelberger, Marcus Hennix, and John Ratcliff. 2007. Position based dynamics. *Journal of Visual Communication and Image Representation* 18, 2 (April 2007), 109–118. https://doi.org/10.1016/j.jvcir.2007.01.005

NVIDIA. [n. d.]. CUDA - Parallel Computing Platform. https://developer.nvidia.com/cuda-zone.

NVIDIA. 2025. NVIDIA FleX. https://developer.nvidia.com/flex

Christoph J Paulus, Lionel Untereiner, Hadrien Courtecuisse, Stéphane Cotin, and David Cazier. 2015. Virtual cutting of deformable objects based on efficient topological operations. *The Visual Computer* 31 (2015), 831–841.

Eftychios Sifakis, Kevin Der, and Ronald Fedkiw. 2007. *Arbitrary cutting of deformable tetrahedralized objects*. https://doi.org/10.1145/1272690.1272701 Journal Abbreviation: Proc. of Symp. on Computer Animation Pages: 80 Publication Title: Proc. of Symp. on Computer Animation.

Denis Steinemann, Miguel A Otaduy, and Markus Gross. 2006. Fast arbitrary splitting of deforming objects. In *Proceedings of the 2006 ACM SIGGRAPH/Eurographics symposium on Computer animation*. 63–72.

Virtual Method Studio. 2025. Obi - Unified Particle Physics for Unity 3D. https://obi.virtualmethodstudio.com/.

Unity. [n. d.]. Unity Physics. https://docs.unity3d.com/Packages/com.unity.physics@1.3/manual/index.html.

Paul Zikas, Antonis Protopsaltis, Nick Lydatakis, Mike Kentros, Stratos Geronikolakis, Steve Kateros, Manos Kamarianakis, Giannis Evangelou, Achilleas Filippidis, Eleni Grigoriou, Dimitris Angelis, Michail Tamiolakis, Michael Dodis, George Kokiadis, John Petropoulos, Maria Pateraki, and George Papagiannakis. 2023. MAGES 4.0: Accelerating the World's Transition to VR Training and Democratizing the Authoring of the Medical Metaverse. *IEEE Computer Graphics and Applications* 43, 2 (2023), 43–56. https://doi.org/10.1109/MCG.2023.3242686